Lab 1 - Introduction to Classes, Objects, and Basic Control Structures

This lab introduces fundamental Java programming concepts through six progressive tasks. Students will learn basic object-oriented programming by creating classes and objects, practice user input handling and control structures, and work with strings and character processing. The exercises progress from building a name management system to implementing a number guessing game, creating a Roman numeral converter, developing a simple grade calculator, building a basic calculator with methods, and implementing a word counting program, establishing core programming skills essential for software development.

Contents

Memorize

Submission Deadline

Deadline to upload the solutions for all tasks is Saturday, 11:59 pm before the lab date.

General Information

The following tasks are to be worked on in fixed teams of two. Each team member must be able to explain all solutions. Please submit only one solution for each team of two. The submission must be a PDF file in our Moodle room with the name and matriculation number. Solutions must be in digital format with intermediate steps and detailed explanations (no handwritten scans). You can use any tool or drawing program of your choice to create the diagrams. If you have questions or need support, use the forum in our Moodle room and help each other.

1. Task 1: Name Management System

This program allows different people to change their name every three years. The program to be created should be able to run on different hardware platforms without recompilation. Therefore, create a Java program that makes this possible. Proceed as follows:

- First, create a class called Person that contains the following attributes: first_name, last_name, name_change_date (separated by day, month, and year).
- Now write a Java program that first outputs the Hamburg greeting for “Good day!” on the console.
- Then your program creates three objects of the Person class, named: `person_1`, `person_2`, and `person_3`.
- Then your program asks which person (1, 2, or 3) wants to change their name.
- Inputting 0 leads to program termination and input 4 displays the stored attributes of all objects. If the input is 1, 2, or 3, it asks for the new first and last name and the day, month, and year from when this change should be valid. This date must be checked regarding the three-year limit. If the check is successful, the change is made in the corresponding object and the result is displayed. If it is not successful, an error message is output and the program continues.
- The inputs in the individual input fields themselves do not need to be checked for correctness, i.e., letters, numbers, special characters, meaning the inputs in the individual input fields must be meaningful and do not need to be checked by your program. In a real project, this preprocessing would be checked by another program and is therefore not the subject of this task.

1.1. Preparation

First clarify the task by drawing the Person class according to UML notation and then creating a structure chart or flowchart that solves this task. Then define all necessary classes in Java.

1.2. Assistance

The following program, which demonstrates console input, can be used to solve this task:

```java
import java.util.Scanner;

public class InputExample {
    public static void main(String[] args) {
```

```
5       // Console input
6       Scanner scanner = new Scanner(System.in);
7
8       System.out.print("Enter something here: ");
9       String input = scanner.nextLine();
10      System.out.println("You entered \"" + input + "\"");
11
12      // If a number is needed for case distinction (e.g., age of majority):
13      System.out.print("Enter your age: ");
14      int age = scanner.nextInt();
15      if (age >= 18) {
16        System.out.println("Of age.");
17      } else {
18        System.out.println("Not yet of age.");
19      }
20
21      scanner.close();
22    }
23  }
```

1.3. Solution

Person.java:

```
1   public class Person {                                                Java
2     String first_name;
3     String last_name;
4     int day;
5     int month;
6     int year;
7
8     // Constructor to initialize a person
9     public Person(String firstName, String lastName, int d, int m, int y) {
10      this.first_name = firstName;
11      this.last_name = lastName;
12      this.day = d;
13      this.month = m;
14      this.year = y;
15    }
16
17    // Display person information
18    public void displayInfo() {
19      System.out.println("Name: " + first_name + " " + last_name);
20      System.out.println("Name change date: " + day + "." + month + "." + year);
```

```java
  }

  // Check if three years have passed since last name change
  public boolean canChangeName(int newDay, int newMonth, int newYear) {
    // Calculate years difference
    int yearsDiff = newYear - this.year;

    // If less than 3 years, return false
    if (yearsDiff < 3) {
      return false;
    }

    // If exactly 3 years, check month and day
    if (yearsDiff == 3) {
      if (newMonth < this.month) {
        return false;
      }
      if (newMonth == this.month && newDay < this.day) {
        return false;
      }
    }

    return true;
  }

  // Change person's name
  public void changeName(String newFirst, String newLast, int d, int m, int y) {
    this.first_name = newFirst;
    this.last_name = newLast;
    this.day = d;
    this.month = m;
    this.year = y;
  }
}
```

NameManagement.java:

```java
import java.util.Scanner;

public class NameManagement {
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    // Hamburg greeting
```

```
8       System.out.println("Moin!");
9
10      // Create three Person objects with initial data
11      Person person_1 = new Person("Max", "Mustermann", 1, 1, 2020);
12      Person person_2 = new Person("Anna", "Schmidt", 15, 6, 2019);
13      Person person_3 = new Person("Peter", "Mueller", 20, 12, 2021);
14
15      boolean running = true;
16
17      while (running) {
18        System.out.println("\n=== Name Management System ===");
19        System.out.println("Which person wants to change their name?");
20        System.out.println("Enter 1, 2, or 3 for the person");
21        System.out.println("Enter 4 to display all persons");
22        System.out.println("Enter 0 to exit");
23        System.out.print("Your choice: ");
24
25        int choice = scanner.nextInt();
26        scanner.nextLine(); // Consume newline
27
28        if (choice == 0) {
29          System.out.println("Program terminated.");
30          running = false;
31        } else if (choice == 4) {
32          // Display all persons
33          System.out.println("\n=== All Persons ===");
34          System.out.println("\nPerson 1:");
35          person_1.displayInfo();
36          System.out.println("\nPerson 2:");
37          person_2.displayInfo();
38          System.out.println("\nPerson 3:");
39          person_3.displayInfo();
40        } else if (choice >= 1 && choice <= 3) {
41          // Select the person to change
42          Person selectedPerson = null;
43          if (choice == 1) selectedPerson = person_1;
44          else if (choice == 2) selectedPerson = person_2;
45          else if (choice == 3) selectedPerson = person_3;
46
47          // Get new name information
48          System.out.print("Enter new first name: ");
49          String newFirst = scanner.nextLine();
50
```

```
51          System.out.print("Enter new last name: ");
52          String newLast = scanner.nextLine();
53
54          System.out.print("Enter day of name change: ");
55          int newDay = scanner.nextInt();
56
57          System.out.print("Enter month of name change: ");
58          int newMonth = scanner.nextInt();
59
60          System.out.print("Enter year of name change: ");
61          int newYear = scanner.nextInt();
62          scanner.nextLine(); // Consume newline
63
64          // Check if three years have passed
65          if (selectedPerson.canChangeName(newDay, newMonth, newYear)) {
66            selectedPerson.changeName(newFirst, newLast, newDay, newMonth,
    newYear);
67            System.out.println("\nName change successful!");
68            selectedPerson.displayInfo();
69          } else {
70            System.out.println("\nError: Three years have not passed since last
    name change!");
71          }
72        } else {
73          System.out.println("Invalid choice!");
74        }
75      }
76
77      scanner.close();
78    }
79  }
```

2. Task 2: Number Guessing Game

Create a Java program that implements a number guessing game where the computer generates a random number and the player tries to guess it. This task will help you practice loops, conditionals, random number generation, and user input handling in Java.

The program should work as follows:

- Generate a random number between 1 and 100 (inclusive)
- Display a welcome message explaining the game rules
- Repeatedly ask the user to guess the number
- For each guess, provide feedback:
  - "Too small!" if the guess is lower than the target number
  - "Too big!" if the guess is higher than the target number
  - "You guessed it!" if the guess is correct
- Keep track of the number of attempts
- When the correct number is guessed, display a congratulatory message and the number of attempts it took
- The program ends when the correct number is guessed

2.1. Requirements

- Use the `Random` class for generating random numbers
- Use `Scanner` class for user input
- Use a loop structure to keep asking for guesses until correct
- Display clear and user-friendly messages
- Assume the user will always enter valid integers (no input validation needed)

2.2. Assistance

The following code snippets demonstrate key concepts needed for this task:

Random number generation:

```java
import java.util.Random;

Random rand = new Random();
int randomNumber = rand.nextInt(100) + 1; // generates 1-100
```

Simple user input with Scanner:

```java
import java.util.Scanner;

Scanner scanner = new Scanner(System.in);
System.out.print("Enter your guess: ");
int userGuess = scanner.nextInt();
```

Basic loop structure:

```java
boolean gameRunning = true;
while (gameRunning) {
    // Game logic here
```

```
4       if (userGuess == randomNumber) {
5           gameRunning = false; // End the game
6       }
7   }
```

2.3. Solution

Java

```java
import java.util.Random;
import java.util.Scanner;

public class NumberGuessingGame {
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    Random random = new Random();

    // Generate random number between 1 and 100
    int targetNumber = random.nextInt(100) + 1;
    int attempts = 0;
    boolean guessedCorrectly = false;

    // Welcome message
    System.out.println("=== Number Guessing Game ===");
    System.out.println("I have chosen a number between 1 and 100.");
    System.out.println("Try to guess it!");

    // Game loop
    while (!guessedCorrectly) {
      System.out.print("\nEnter your guess: ");
      int userGuess = scanner.nextInt();
      attempts++;

      // Check the guess
      if (userGuess < targetNumber) {
        System.out.println("Too small!");
      } else if (userGuess > targetNumber) {
        System.out.println("Too big!");
      } else {
        System.out.println("You guessed it!");
        guessedCorrectly = true;
      }
    }

    // Display final message
```

```
37      System.out.println("\nCongratulations! You found the number " +
    targetNumber);
38      System.out.println("It took you " + attempts + " attempts.");
39
40      scanner.close();
41    }
42  }
```

3. Task 3: Roman Numeral Converter

Create a Java program that converts Roman numerals to integers. This task will help you practice working with strings, character processing, and conditional logic.

Roman numerals use seven different symbols with specific values:

- I = 1
- V = 5
- X = 10
- L = 50
- C = 100
- D = 500
- M = 1000

3.1. How Roman Numerals Work

Most of the time, Roman numerals are written from largest to smallest (left to right) and you simply add up the values:

- “III” = 1 + 1 + 1 = 3
- “XII” = 10 + 1 + 1 = 12
- “LVIII” = 50 + 5 + 1 + 1 + 1 = 58

However, there are special cases where a smaller numeral appears before a larger one, meaning you subtract instead of add:

- “IV” = 5 - 1 = 4 (not “IIII”)
- “IX” = 10 - 1 = 9
- “XL” = 50 - 10 = 40
- “XC” = 100 - 10 = 90
- “CD” = 500 - 100 = 400
- “CM” = 1000 - 100 = 900

3.2. Your Task

Write a program that:

1. Asks the user to enter a Roman numeral as a string
2. Converts it to an integer using the rules above
3. Displays the result

Test your program with these examples:

- “III” should give 3
- “IV” should give 4
- “IX” should give 9
- “LVIII” should give 58
- “MCMXCIV” should give 1994

3.3. Requirements

- Use `Scanner` for input
- Process the string character by character
- Handle both normal addition and subtraction cases
- Display clear output

3.4. Assistance

Getting individual characters from a string:

```java
String roman = "XIV";
char firstChar = roman.charAt(0);  // Gets 'X'
int length = roman.length();       // Gets 3
```

Simple approach - check current and next character:

```java
String roman = "IV";
int total = 0;

for (int i = 0; i < roman.length(); i++) {
    char current = roman.charAt(i);

    // Check if we need to subtract (when current < next)
    if (i < roman.length() - 1) {
        char next = roman.charAt(i + 1);
        // Add your logic here to compare current and next
    }
}
```

Getting the value of a Roman numeral character:

```java
// You can use if-else statements or a switch
int getValue(char c) {
    if (c == 'I') return 1;
    if (c == 'V') return 5;
    if (c == 'X') return 10;
    // ... continue for other characters
    return 0;
}
```

3.5. Solution

```java
import java.util.Scanner;

public class RomanNumeralConverter {
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    System.out.println("=== Roman Numeral Converter ===");
    System.out.print("Enter a Roman numeral: ");
    String roman = scanner.nextLine().toUpperCase();

    int result = convertRomanToInteger(roman);
```

```
12      System.out.println(roman + " = " + result);
13
14      scanner.close();
15    }
16
17    // Convert a Roman numeral character to its integer value
18    public static int getValue(char romanChar) {
19      switch (romanChar) {
20        case 'I': return 1;
21        case 'V': return 5;
22        case 'X': return 10;
23        case 'L': return 50;
24        case 'C': return 100;
25        case 'D': return 500;
26        case 'M': return 1000;
27        default: return 0;
28      }
29    }
30
31    // Convert a full Roman numeral string to an integer
32    public static int convertRomanToInteger(String roman) {
33      int total = 0;
34
35      for (int i = 0; i < roman.length(); i++) {
36        int currentValue = getValue(roman.charAt(i));
37
38        // Check if we need to subtract (when current < next)
39        if (i < roman.length() - 1) {
40          int nextValue = getValue(roman.charAt(i + 1));
41
42          if (currentValue < nextValue) {
43            // Subtraction case (e.g., IV, IX, XL)
44            total -= currentValue;
45          } else {
46            // Normal addition
47            total += currentValue;
48          }
49        } else {
50          // Last character, always add
51          total += currentValue;
52        }
53      }
54
```

```
55      return total;
56    }
57  }
```

4. Task 4: Grade Manager

Create a Java program that calculates and manages student grades. This task will help you practice working with arrays, loops, mathematical operations, and basic data processing in Java.

Your program should implement a simple grade management system that:

- Stores grades for multiple students
- Calculates statistics like average, highest, and lowest grades
- Determines letter grades based on numerical scores
- Provides a summary report

4.1. Program Requirements

Your program should:

1. Ask the user how many students they want to enter grades for
2. Create an array to store the grades
3. Prompt the user to enter each student's grade (0-100)
4. Calculate and display:
    - The average grade
    - The highest grade
    - The lowest grade
    - How many students passed (grade >= 60)
    - How many students failed (grade < 60)
5. Display each student's numerical grade along with their letter grade

4.2. Letter Grade Scale

Use the following grading scale:

- A: 90-100
- B: 80-89
- C: 70-79
- D: 60-69
- F: 0-59

4.3. Requirements

- Use an array to store the grades
- Use loops to process the data
- Use the `Scanner` class for user input
- Display results in a clear, formatted way
- Assume the user will always enter valid numbers between 0-100

4.4. Assistance

Creating and working with arrays:

```java
// Create an array to hold grades
int numberOfStudents = 5;
double[] grades = new double[numberOfStudents];

// Store a grade in the array
grades[0] = 85.5;  // First student's grade
```

```
7
8 // Get a grade from the array
9 double firstGrade = grades[0];
```

Sample output format:

```
1  === Grade Report ===
2  Student 1: 85.5 (B)
3  Student 2: 92.0 (A)
4  Student 3: 78.5 (C)
5  Student 4: 67.0 (D)
6  Student 5: 88.5 (B)
7
8  === Statistics ===
9  Average Grade: 82.3
10 Highest Grade: 92.0
11 Lowest Grade: 67.0
12 Students Passed: 5
13 Students Failed: 0
```

4.5. Solution

Java

```java
import java.util.Scanner;

public class GradeManager {
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    // Ask for number of students
    System.out.print("How many students do you want to enter grades for? ");
    int numStudents = scanner.nextInt();

    // Create array to store grades
    double[] grades = new double[numStudents];

    // Input grades
    for (int i = 0; i < numStudents; i++) {
      System.out.print("Enter grade for student " + (i + 1) + ": ");
      grades[i] = scanner.nextDouble();
    }

    // Calculate statistics
    double sum = 0;
    double highest = grades[0];
    double lowest = grades[0];
```

```
24      int passed = 0;
25      int failed = 0;
26
27      for (int i = 0; i < numStudents; i++) {
28        sum += grades[i];
29
30        if (grades[i] > highest) {
31          highest = grades[i];
32        }
33
34        if (grades[i] < lowest) {
35          lowest = grades[i];
36        }
37
38        if (grades[i] >= 60) {
39          passed++;
40        } else {
41          failed++;
42        }
43      }
44
45      double average = sum / numStudents;
46
47      // Display grade report
48      System.out.println("\n=== Grade Report ===");
49      for (int i = 0; i < numStudents; i++) {
50        String letterGrade = getLetterGrade(grades[i]);
51        System.out.println("Student " + (i + 1) + ": " + grades[i] + " (" +
letterGrade + ")");
52      }
53
54      // Display statistics
55      System.out.println("\n=== Statistics ===");
56      System.out.println("Average Grade: " + average);
57      System.out.println("Highest Grade: " + highest);
58      System.out.println("Lowest Grade: " + lowest);
59      System.out.println("Students Passed: " + passed);
60      System.out.println("Students Failed: " + failed);
61
62      scanner.close();
63    }
64
65    // Convert numerical grade to letter grade
```

```
66    public static String getLetterGrade(double grade) {
67      if (grade >= 90) {
68        return "A";
69      } else if (grade >= 80) {
70        return "B";
71      } else if (grade >= 70) {
72        return "C";
73      } else if (grade >= 60) {
74        return "D";
75      } else {
76        return "F";
77      }
78    }
79  }
```

5. Task 5: Simple Calculator with Methods

Create a Java program that implements a basic calculator using methods. This task will help you practice method creation, parameter passing, return values, and program organization in Java.

Your calculator should be able to perform basic arithmetic operations (addition, subtraction, multiplication, division) and be organized using separate methods for each operation.

5.1. Program Requirements

Your program should:

1. Display a menu of available operations to the user
2. Ask the user to select an operation
3. Prompt for two numbers
4. Perform the calculation using the appropriate method
5. Display the result
6. Allow the user to perform multiple calculations
7. Exit when the user chooses to quit

5.2. Menu Options

```
1 === Simple Calculator ===
2 1. Addition (+)
3 2. Subtraction (-)
4 3. Multiplication (*)
5 4. Division (/)
6 5. Exit
7 Choose an operation (1-5):
```

5.3. Requirements

- Create separate methods for each arithmetic operation
- Each method should take two parameters and return the result
- Handle division by zero appropriately
- Use a loop to allow multiple calculations
- Use the `Scanner` class for user input
- Display results in a clear format

5.4. Assistance

Sample program execution:

```
1  === Simple Calculator ===
2  1. Addition (+)
3  2. Subtraction (-)
4  3. Multiplication (*)
5  4. Division (/)
6  5. Exit
7  Choose an operation (1-5): 1
8  Enter first number: 15.5
9  Enter second number: 8.3
```

```
Result: 15.5 + 8.3 = 23.8

=== Simple Calculator ===
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
5. Exit
Choose an operation (1-5): 4
Enter first number: 10
Enter second number: 0
Error: Division by zero!
Result: 10.0 / 0.0 = 0.0

=== Simple Calculator ===
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
5. Exit
Choose an operation (1-5): 5
Thank you for using the calculator!
```

5.5. Solution

Java

```java
import java.util.Scanner;

public class SimpleCalculator {
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    boolean running = true;

    while (running) {
      // Display menu
      System.out.println("\n=== Simple Calculator ===");
      System.out.println("1. Addition (+)");
      System.out.println("2. Subtraction (-)");
      System.out.println("3. Multiplication (*)");
      System.out.println("4. Division (/)");
      System.out.println("5. Exit");
      System.out.print("Choose an operation (1-5): ");

      int choice = scanner.nextInt();

```

```
20        // Exit if user chooses 5
21        if (choice == 5) {
22          System.out.println("Thank you for using the calculator!");
23          running = false;
24          continue;
25        }
26
27        // Get numbers from user
28        System.out.print("Enter first number: ");
29        double num1 = scanner.nextDouble();
30
31        System.out.print("Enter second number: ");
32        double num2 = scanner.nextDouble();
33
34        // Perform operation based on choice
35        double result = 0;
36        String operation = "";
37
38        switch (choice) {
39          case 1:
40            result = add(num1, num2);
41            operation = "+";
42            break;
43          case 2:
44            result = subtract(num1, num2);
45            operation = "-";
46            break;
47          case 3:
48            result = multiply(num1, num2);
49            operation = "*";
50            break;
51          case 4:
52            if (num2 == 0) {
53              System.out.println("Error: Division by zero!");
54              result = 0;
55            } else {
56              result = divide(num1, num2);
57            }
58            operation = "/";
59            break;
60          default:
61            System.out.println("Invalid choice!");
62            continue;
```

```
63        }
64
65        // Display result
66        System.out.println("Result: " + num1 + " " + operation + " " + num2 + " = " + result);
67      }
68
69      scanner.close();
70    }
71
72    // Method for addition
73    public static double add(double a, double b) {
74      return a + b;
75    }
76
77    // Method for subtraction
78    public static double subtract(double a, double b) {
79      return a - b;
80    }
81
82    // Method for multiplication
83    public static double multiply(double a, double b) {
84      return a * b;
85    }
86
87    // Method for division
88    public static double divide(double a, double b) {
89      return a / b;
90    }
91  }
```

6. Task 6: Word Counter Program

Create a Java program that analyzes text input and counts various statistics about words and characters. This task will help you practice string manipulation, character analysis, and data processing in Java.

Your program should analyze a sentence or paragraph entered by the user and provide detailed statistics about the text.

6.1. Program Requirements

Your program should:

1. Ask the user to enter a sentence or paragraph
2. Count and display:
   - Total number of characters (including spaces)
   - Total number of characters (excluding spaces)
   - Total number of words
   - Total number of sentences (based on periods, exclamation marks, question marks)
   - Number of vowels (a, e, i, o, u - case insensitive)
   - Number of consonants
   - The longest word in the text
3. Display all results in a clear, formatted report

6.2. Requirements

- Use string methods for text processing
- Handle both uppercase and lowercase letters
- Consider punctuation appropriately
- Display results in a well-formatted report
- Use the `Scanner` class for user input

6.3. Assistance

Character analysis loop:

```java
String text = "Hello World!";
int vowelCount = 0;
int consonantCount = 0;
int totalChars = text.length();
int charsWithoutSpaces = 0;

for (int i = 0; i < text.length(); i++) {
    char c = text.charAt(i);

    if (Character.isLetter(c)) {
        charsWithoutSpaces++;
        if (isVowel(c)) {
            vowelCount++;
        } else {
            consonantCount++;
```

```
        }
    } else if (c != ' ') {
        charsWithoutSpaces++; // Count non-space, non-letter characters
    }
}
```

Sample program output:

```
Enter a sentence or paragraph to analyze:
Hello world! This is a great example of text analysis. How cool is that?

=== TEXT ANALYSIS REPORT ===
Original text: "Hello world! This is a great example of text analysis. How cool is that?"

Character Statistics:
- Total characters (with spaces): 78
- Total characters (without spaces): 64
- Number of vowels: 24
- Number of consonants: 40

Word Statistics:
- Total number of words: 14
- Longest word: "analysis"

Sentence Statistics:
- Total number of sentences: 2
```

6.4. Solution

Java

```java
import java.util.Scanner;

public class WordCounter {
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    System.out.println("Enter a sentence or paragraph to analyze:");
    String text = scanner.nextLine();

    // Initialize counters
    int totalChars = text.length();
    int charsWithoutSpaces = 0;
    int vowelCount = 0;
    int consonantCount = 0;
    int wordCount = 0;
```

```
16      int sentenceCount = 0;
17      String longestWord = "";
18
19      // Count characters, vowels, and consonants
20      for (int i = 0; i < text.length(); i++) {
21        char c = text.charAt(i);
22
23        // Count characters without spaces
24        if (c != ' ') {
25          charsWithoutSpaces++;
26        }
27
28        // Count vowels and consonants
29        if (Character.isLetter(c)) {
30          char lowerChar = Character.toLowerCase(c);
31          if (isVowel(lowerChar)) {
32            vowelCount++;
33          } else {
34            consonantCount++;
35          }
36        }
37
38        // Count sentences (., !, ?)
39        if (c == '.' || c == '!' || c == '?') {
40          sentenceCount++;
41        }
42      }
43
44      // Count words and find longest word
45      String[] words = text.split("\\s+");
46      wordCount = words.length;
47
48      for (String word : words) {
49        // Remove punctuation from word for comparison
50        String cleanWord = word.replaceAll("[^a-zA-Z]", "");
51
52        if (cleanWord.length() > longestWord.length()) {
53          longestWord = cleanWord;
54        }
55      }
56
57      // Display results
58      System.out.println("\n=== TEXT ANALYSIS REPORT ===");
```

```
59      System.out.println("Original text: \"" + text + "\"");
60
61      System.out.println("\nCharacter Statistics:");
62      System.out.println("- Total characters (with spaces): " + totalChars);
63      System.out.println("- Total characters (without spaces): " +
    charsWithoutSpaces);
64      System.out.println("- Number of vowels: " + vowelCount);
65      System.out.println("- Number of consonants: " + consonantCount);
66
67      System.out.println("\nWord Statistics:");
68      System.out.println("- Total number of words: " + wordCount);
69      System.out.println("- Longest word: \"" + longestWord + "\"");
70
71      System.out.println("\nSentence Statistics:");
72      System.out.println("- Total number of sentences: " + sentenceCount);
73
74      scanner.close();
75    }
76
77    // Check if a character is a vowel
78    public static boolean isVowel(char c) {
79      return c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u';
80    }
81  }
```

7. Lab Execution

If your program is not yet working without issue, we will try to correct this during the course of the lab. With good preparation, this should not be a problem. Every student is required to be able to explain their thought process at the beginning of the lab. By the end of the lab, the task needs to be completed. Of course, we will support you, but your personal commitment must also be clearly recognizable! Julian Moldenhauer, Furkan Yildirim, and Emily Antosch wish you lots of fun and success!